



第12课时：一维数组

存储 · 访问 · 遍历

数组的概念、声明、初始化与遍历



为什么需要数组？



在编程中，我们经常需要处理**大量相同类型的数据**，比如：

- 一个班 50 名学生的成绩
- 一组实验数据（温度、湿度等）
- 一个游戏中的排行榜

如果为每个数据单独定义变量（`score1`，`score2`，...），代码会极其冗长且难以维护。

数组正是为了解决这类问题而生的——它允许我们用**一个名字**管理**一组连续**的相同类型数据。



数组的概念



数组 (Array) 是相同数据类型的元素的有序集合，存储在连续的内存空间中。

- **所有元素类型相同**（例如全是 `int` 或全是 `double`）
- **通过下标（索引）访问每个元素**
- **下标从 0 开始，这是 C++ 的重要规则**

“ 可以把数组想象成一排编号的储物柜，每个柜子只能放同一种物品，编号从 0 开始。

”



数组的声明



声明数组需要指定：**类型**、**数组名**、**元素个数**（必须为常量表达式）。

// 语法

类型 数组名[元素个数];

// 示例

int scores[50]; // 50 个 **int** 类型元素，下标 0~49

double temperatures[7]; // 7 个 **double** 类型元素

char letters[26]; // 26 个 **char** 类型元素

注意：元素个数必须是**编译期常量**，不能是变量（除非使用动态数组，后续学习）。

数组的初始化



可以在声明时直接初始化数组。

方式一：完全初始化

```
int arr[5] = {10, 20, 30, 40, 50};
```

方式二：部分初始化（未指定的元素自动初始化为 0）

```
int arr[5] = {10, 20}; // arr[0]=10, arr[1]=20, arr[2]=0, arr[3]=0, arr[4]=0
```

方式三：省略大小（编译器根据初始化列表自动推断）

```
int arr[] = {1, 2, 3, 4, 5}; // 大小为 5
```

方式四：全部初始化为 0

```
int arr[100] = {}; // 所有元素为 0  
// 或  
int arr[100] = {0};
```

访问数组元素



通过 **数组名[下标]** 来访问或修改元素，下标从 **0** 开始。

```
int arr[5] = {10, 20, 30, 40, 50};

cout << arr[0];    // 输出 10 (第一个元素)
cout << arr[4];    // 输出 50 (最后一个元素)

arr[2] = 100;      // 将第三个元素改为 100
```

重要：访问下标超出合法范围（0 ~ 大小-1）会导致**未定义行为**，可能崩溃或产生错误数据，务必小心。

数组下标为什么从 0 开始?



这是 C/C++ 从 C 语言继承来的设计，与内存地址计算有关：

- 数组名代表数组**首元素的内存地址**
- `arr[i]` 等价于 `*(arr + i)`，即首地址偏移 `i` 个元素大小
- 如果从 1 开始，每次访问都需要做 `*(arr + i - 1)`，多了减法开销

虽然对现代编程影响不大，但这是 C++ 的固定规则，必须记住：**第一个元素下标是 0，最后一个是大-1。**

数组的遍历



遍历 (Traversal) 是指依次访问数组中的每一个元素。最常用的是 `for` 循环。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    for (int i = 0; i < 5; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
// 输出: 10 20 30 40 50
```

例题1：数组输入与输出



题目：输入 n 个整数 ($n \leq 100$)，将它们存储到数组中，然后逆序输出。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    int arr[100];    // 最大容量 100

    // 输入
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // 逆序输出
    for (int i = n - 1; i >= 0; i--) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

例题2：求数组中的最大值



思路：假设第一个元素最大，然后依次比较后面的元素，更新最大值。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    int arr[100];
    for (int i = 0; i < n; i++) cin >> arr[i];

    int maxVal = arr[0];    // 初始化为第一个元素
    for (int i = 1; i < n; i++) {
        if (arr[i] > maxVal) {
            maxVal = arr[i];
        }
    }
    cout << "最大值是: " << maxVal << endl;
    return 0;
}
```

例题3：统计数组中某个值的出现次数



题目：统计数组中有多少个元素等于目标值 target。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int arr[10] = {2, 5, 3, 2, 8, 2, 1, 7, 2, 4};
    int target = 2;
    int count = 0;
    for (int i = 0; i < 10; i++) {
        if (arr[i] == target) {
            count++;
        }
    }
    cout << target << " 出现了 " << count << " 次" << endl;
    return 0;
}
// 输出: 2 出现了 4 次
```

数组与循环的常见模式



模式	代码示例
遍历并输出	<pre>for (i=0; i<n; i++) cout << a[i];</pre>
累加求和	<pre>sum += a[i];</pre>
查找最大值/最小值	<pre>if (a[i] > max) max = a[i];</pre>
统计满足条件的个数	<pre>if (a[i] % 2 == 0) cnt++;</pre>
逆序输出	<pre>for (i=n-1; i>=0; i--) cout << a[i];</pre>
数组元素赋值（输入）	<pre>cin >> a[i];</pre>
数组初始化	<pre>int a[5] = {1,2,3,4,5};</pre>

错误类型	示例/说明	解决方法
数组下标越界	<code>arr[5]</code> 当大小为 5 时（合法下标 0~4）	检查循环条件，确保 <code><</code> <code>大小</code>
使用未初始化的数组元素	局部数组未初始化，元素包含随机值	声明时初始化或手动赋值
数组大小使用变量（非 <code>const</code> ）	<code>int n; cin>>n; int a[n];</code> （部分编译器支持但非标准）	用常量或用动态数组（后续）
误以为数组名可以直接赋值	<code>arr2 = arr1;</code> 是非法的	用循环逐个元素复制
省略大小时未全部初始化	<code>int a[] = {1,2};</code> 大小是 2，没问题，但注意推断	明确写出大小或确保所有元素列出

调试技巧：在循环中打印 `i` 和 `arr[i]`，检查是否按预期访问。

数组在内存中的存储



数组元素在内存中是**连续存放**的，每个元素占用相同大小的字节数。

例如 `int arr[5]` 的内存布局（假设 int 占 4 字节）：

“ 知道这个特性有助于理解指针和数组的关系（后续学习）。”

一维数组的典型应用



- **数据存储**：保存一批输入数据以便后续处理（如排序、统计）
- **查找与搜索**：线性查找、二分查找（后续）
- **序列处理**：求最大/最小、求和、平均值
- **标记数组**：用 bool 数组标记某些状态（如素数筛）
- **计数数组**：统计各分数段人数

数组是算法实现中最基础、最常用的数据结构。

总结



- **数组**是相同类型数据的**连续集合**，通过**下标**访问，下标从 **0** 开始。
- 声明： `类型 数组名[大小];`
- 初始化：可以在声明时用 `{}` 赋值，未指定的自动归零。
- 遍历：常用 `for` 循环，从 `0` 到 `大小-1`。
- 边界：**绝不越界**，否则程序可能崩溃。
- 数组操作的核心是循环，掌握数组就能处理批量数据。

下节课我们将深入学习**数组排序**，从冒泡排序开始，理解算法思想。

谢谢大家！



数组让批量数据处理变得简单高效！

Q & A